EL JOKER 2.0

Claude Code Full Handoff — Build From Scratch

Chain619 · Hedera Hackathon 2026

Autonomous, but never uncontrolled.

Purpose: give a fresh Claude Code session enough product, technical, security, UX and demo context to build the complete project from zero without relying on previous conversation history.

1. Executive Product Brief

EL JOKER is a verifiable AI-powered emergency procurement and spending-governance platform. An operator describes an urgent mission. AI helps research and rank suppliers. Independent verification checks the evidence. A deterministic Guardian enforces policy. High-risk actions require a human. Hedera creates public, tamper-evident proofs. Payment executes only when authorization is valid.

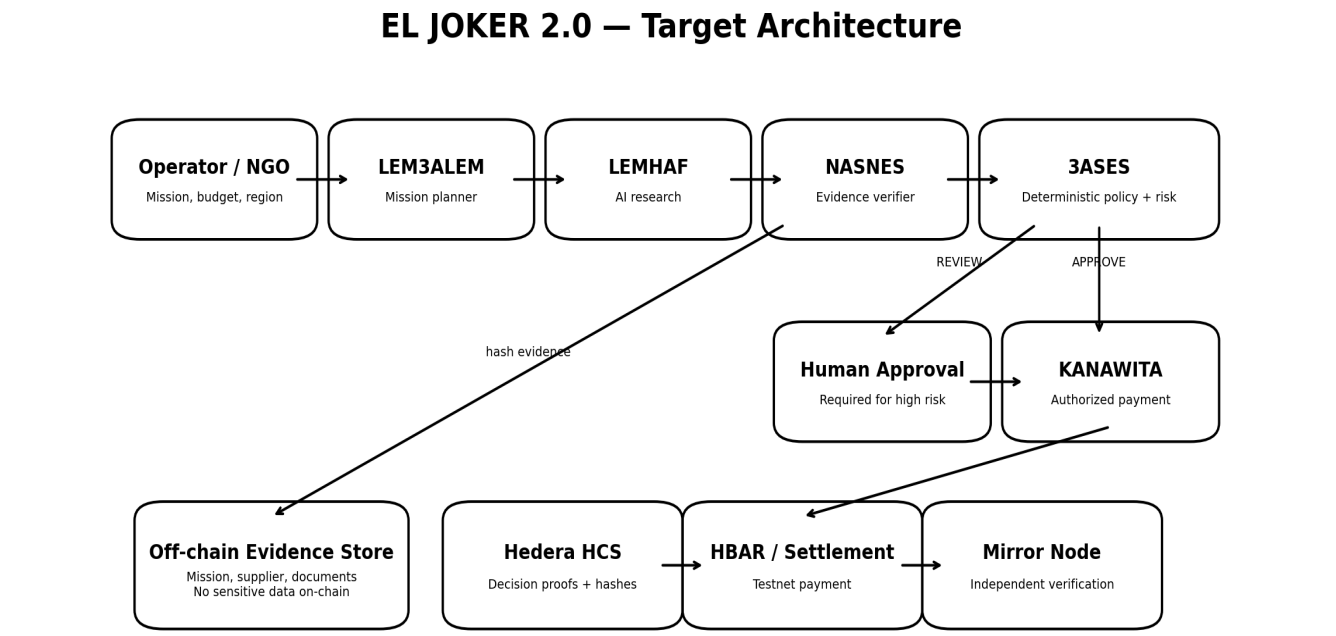
The winning thesis is not “five AI agents.” It is **safe autonomous spending with independent proof**.

Primary demo: 500 people in Gabès need water and essential medicine under a fixed budget. The system must show both successful execution and credible refusal modes.

2. What must remain unique

Agent	Role	Non-negotiable property
LEM3ALEM	Coordinator / mission planner	Turns request into validated mission
LEMHAF	AI research	May reason and rank; never authorizes money
NASNES	Verifier	Independently checks supplier/evidence
3ASES	Guardian	Deterministic rules only; no LLM
KANAWITA	Payment	Cannot execute without authorization

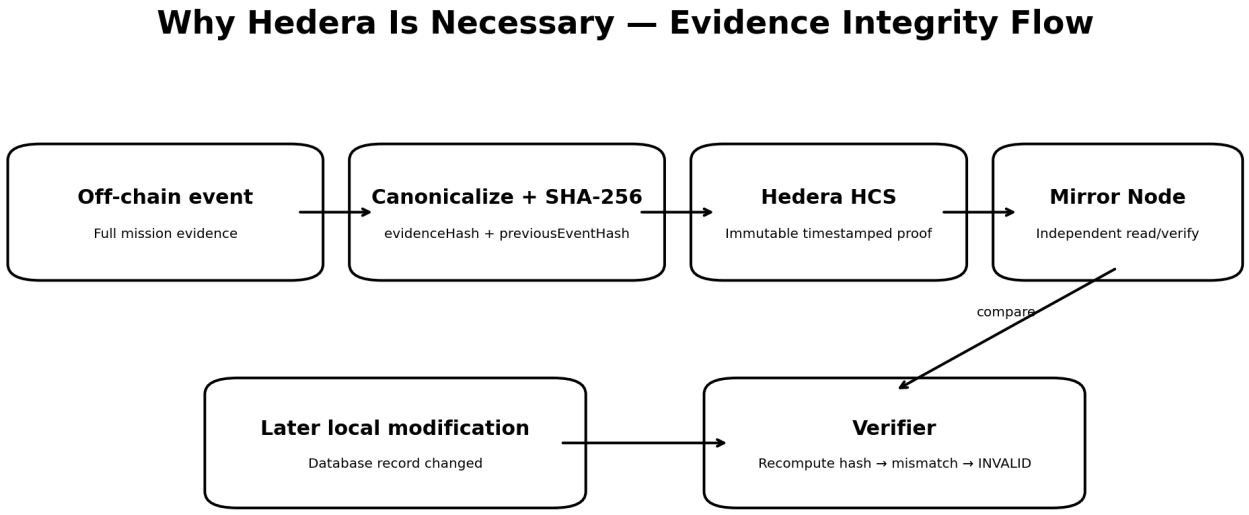
3. Target architecture



Core thesis: AI proposes → verification checks → deterministic policy decides → humans control risk → Hedera proves → payment executes only when authorized.

Recommended implementation: TypeScript end-to-end; Next.js frontend; Node/TypeScript API; Zod validation; simple persistent DB; Anthropic SDK for the real AI path; Hedera JS SDK for testnet HCS and payment. Exact libraries may be adjusted to current official SDK guidance.

4. Why Hedera must matter



A normal app can edit its own history. EL JOKER can prove that the current evidence still matches the public consensus record.

Do not use blockchain as decoration. The strongest proof is a tamper demo: alter local evidence in development mode, recompute the evidence hash, compare it with the Hedera consensus proof, and show INVALID / TAMPERING DETECTED. Only hashes and minimal proof metadata go on-chain; operational and sensitive data remain off-chain.

5. Required policy engine

3ASES must be a versioned deterministic policy engine. Minimum checks: budget, allowed region, verified supplier, duplicate payment, duplicate invoice, price anomaly, category restriction, autonomous transaction limit, valid mission state, Nasnes verification, and human-approval threshold. Decisions: APPROVED, BLOCKED, HUMAN_REVIEW_REQUIRED. Risk levels: LOW, MEDIUM, HIGH, CRITICAL.

This separation is a core security argument: probabilistic AI can recommend, but authorization is reproducible and auditable.

6. Six live demo scenarios

Winning Live Demo — 6 Scenarios

A. Approved mission

Verified supplier + valid price → pay → proof

B. Budget block

Offer exceeds budget → BLOCK → 0 payment

C. Duplicate attack

First payment succeeds → second attempt blocked

D. Fake supplier

AI recommends it → Nasnes/3ASES reject it

E. Human review

High-risk amount → operator approval required

F. Tamper detection

Local history changed → Hedera proof mismatch

Final screen: Mission audit timeline + HashScan/Mirror verification + clear APPROVED/BLOCKED/REVIEW status.

7. UI direction

Preserve the Tunisian personality and five characters. The experience should feel like a mission-control group chat, not a generic banking dashboard. The main screen must make current agent, mission, budget, supplier, risk, Guardian decision, human approval, payment and Hedera proof status visible without navigating through many pages.

Required status chips: AI ANALYZING · VERIFIED · APPROVED · BLOCKED · HUMAN REVIEW · PAID · PROOF VERIFIED.

Required proof screen: LOCAL RECORD vs HEDERA PROOF vs VERIFICATION RESULT, with HashScan link when the proof is real.

8. Security and honesty requirements

- Testnet only. Never mainnet for the hackathon demo.
- Never commit private keys or API keys; provide .env.example only.
- Never claim a real AI call when fallback/demo mode ran.
- Never expose chain-of-thought; keep only concise output summaries and execution metadata.
- Never put beneficiary personal data or full supplier documents on Hedera.
- Never mark a payment successful before network confirmation.
- Use idempotency to make duplicate payment impossible.
- Do not invent NGO customers, pilots, partnerships, field validation or legal approval.
- Do not add tokens or smart contracts unless they solve a real product requirement.

9. Product / business framing

Target users: NGOs, humanitarian organizations, emergency-response programs, grant/disbursement programs. **Value:** fast AI-assisted procurement without uncontrolled autonomous spending. **Possible model:** B2B SaaS, organization subscription, usage-based proof volume, or enterprise deployment. These are hypotheses, not validated customers.

10. Recommended build order

Phase	Area	Deliverable
1	Scaffold	Monorepo/app structure, domain models, DB, UI shell
2	Mission	LEM3ALEM + seeded supplier/offer demo data
3	AI + verification	LEMHAF real/fallback modes + NASNES
4	Policy	3ASES rules, risk, versioned policy
5	Approval	Human review/reject/approve flow
6	Proof	HCS events, canonical hashing, Mirror verification
7	Payment	Safe testnet transfer + idempotency
8	Demo	Six scenarios
9	QA	Unit/integration/E2E tests + failure handling
10	Polish	Audit timeline, proof UI, README, demo script

11. Acceptance gate

Claude Code should not claim completion until the app starts, tests pass, all six scenarios work, real AI is distinguishable from fallback, 3ASES has no LLM, Hedera HCS proofs are verifiable, Mirror Node handling works,

testnet payment is confirmed, duplicate payment is blocked, human review stops payment, and local tampering is detected.

12. What NOT to build

Avoid feature bloat: no unnecessary token, marketplace, DAO, complex microservices, production identity stack, or smart contract solely for presentation. The jury should understand the security thesis in under one minute and see it proven in a few minutes.

13. Official references for Claude Code

Use current official documentation during implementation because SDK versions and APIs change.

Hedera docs: https://docs.hedera.com/

Hedera docs index for coding assistants: https://docs.hedera.com/llms.txt

Hedera developer portal: https://portal.hedera.com/

HashScan: https://hashscan.io/

Scheduled transactions: https://docs.hedera.com/hedera/core-concepts/scheduled-transaction

Anthropic docs: https://docs.anthropic.com/

Claude Code CLI: https://docs.anthropic.com/en/docs/claude-code/cli-usage

Claude prompting best practices:
https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/prompt-templates-and-variables

14. First message to Claude Code

Read MASTER_PROMPT_CLAUDE_CODE.md and this PDF completely. Inspect the architecture images. Build the project from scratch in this repository. Do not only plan: implement phase by phase, run tests after each phase, fix failures before continuing, and keep PROJECT_STATUS.md updated with completed work, decisions, failures, credentials still required, and next steps.